



Examen 16 Junio 2016 con soluciones

Estructuras de Datos Avanzadas (Universidad Rey Juan Carlos)



Scan to open on Studocu

Ejercicio 1 [3.0 puntos].

1.1: [1.0 punto] Represente gráficamente los árboles AVL y Rojo-Negor de altura 5 que almacenen el menor número posible de elementos.

1.2: [1.0 punto] Se quiere utilizar un mapa para almacenar, como máximo 50 entradas. Determine y justifique qué tamaño debería tener dicha tabla si las colisiones se resuelven por encadenamiento separado y por direccionamiento abierto.

1.3: [1.0 punto] Determine y justifique el número máximo de claves que se puede almacenar en un árbol B de orden 5 y altura 3.

Ejercicio 2 [3.5 puntos]

Se desea aumentar la funcionalidad de los árboles binarios de búsqueda incorporando la siguiente funcionalidad:

- a) [1.0 punto] Implementar método `toLinkedList`, que crea un `LinkedList` con la misma información y la misma relación padre hijo que el `LinkedBinarySearchTree` de partida.
- b) [1.5 puntos] Implementar el método `removeRange`, que recibe dos argumentos comparables, `[min, max]`, y eliminan los nodos del árbol cuyo valor esté dentro de dicho rango.
- c) [1.0 punto] Dada la siguiente clase `Student`, hacer los cambios e implementar los métodos que se considere oportunos sobre ella para que se puedan insertar en `LinkedBinarySearchTree` objetos de dicha clase ordenados por el número de expediente.

```
public class Student{  
    String name;  
    String surname;  
    int exp;  
  
    public Student(){  
    }  
}
```

NOTA: Se valorará un buen diseño (de clases y métodos) para obtener la máxima calificación. Se recuerda que, por lo general, es un mal diseño tener métodos que compartan mucho código. Asimismo, es imprescindible que los métodos implementados sean lo más *eficientes* y *genéricos* posible.

Ejercicio 3 [3.5 puntos]

Recientemente la URJC ha conseguido el contrato para gestionar el censo de los habitantes de Madrid de cara a las próximas elecciones. Cada votante tiene un número de DNI y, asociado a dicho número, aparece el colegio electoral y la mesa en la que debe votar.

Cada colegio da servicio a un conjunto de calles de cada municipio. A su vez, cada colegio tiene varias mesas identificadas con letras del alfabeto. Por último, cada mesa da servicio a un máximo de 800 personas. Por ejemplo, si un colegio da servicio a la calles: Rosal, Margarita y Lirio, con 701, 849 y 377 personas cada una, harán falta 3 mesas (la A para 643 votantes, la B para 642 votantes y la C para 642 votantes).

Además, los votantes se reparten entre las mesas de manera ordenada por DNI. Es decir, la mesa A tiene los votantes con número de DNI más bajo, la mesa B tiene los siguientes en numeración, y así sucesivamente.

La información llega a los colegios en cadenas de texto como la siguiente:

645342334H, c/ Clavel, Manolo Gutiérrez Cano

532452344H, c/ Margarita, Roberto Pérez Alonso

Se desea almacenar toda esta información en varias estructuras de datos para que se puedan realizar dos tipos de consultas. Por un lado, que un votante, introduciendo su DNI pueda buscar la mesa que le corresponde. Por otro lado, se desea crear el listado de los votantes de una mesa que debe estar ordenado por número de DNI creciente.

Se pide:

- [1.0 punto]** Implementar dentro de la clase `ElectoralCollege` las estructuras de datos que se consideren convenientes para cumplir con los requisitos indicados.
- [1.0 punto]** Implementar los métodos de inicialización `addVoter` (para alimentar la clase con los votantes posibles) y el método `makeStationDistribution` (que reparte los votantes almacenados entre las diferentes mesas electorales).
- [1.5 puntos]** Implementar los métodos de consulta `getAllVoters` (para obtener el censo total del colegio), y el método `getStationVoters` (para obtener votantes de cada mesa electoral).

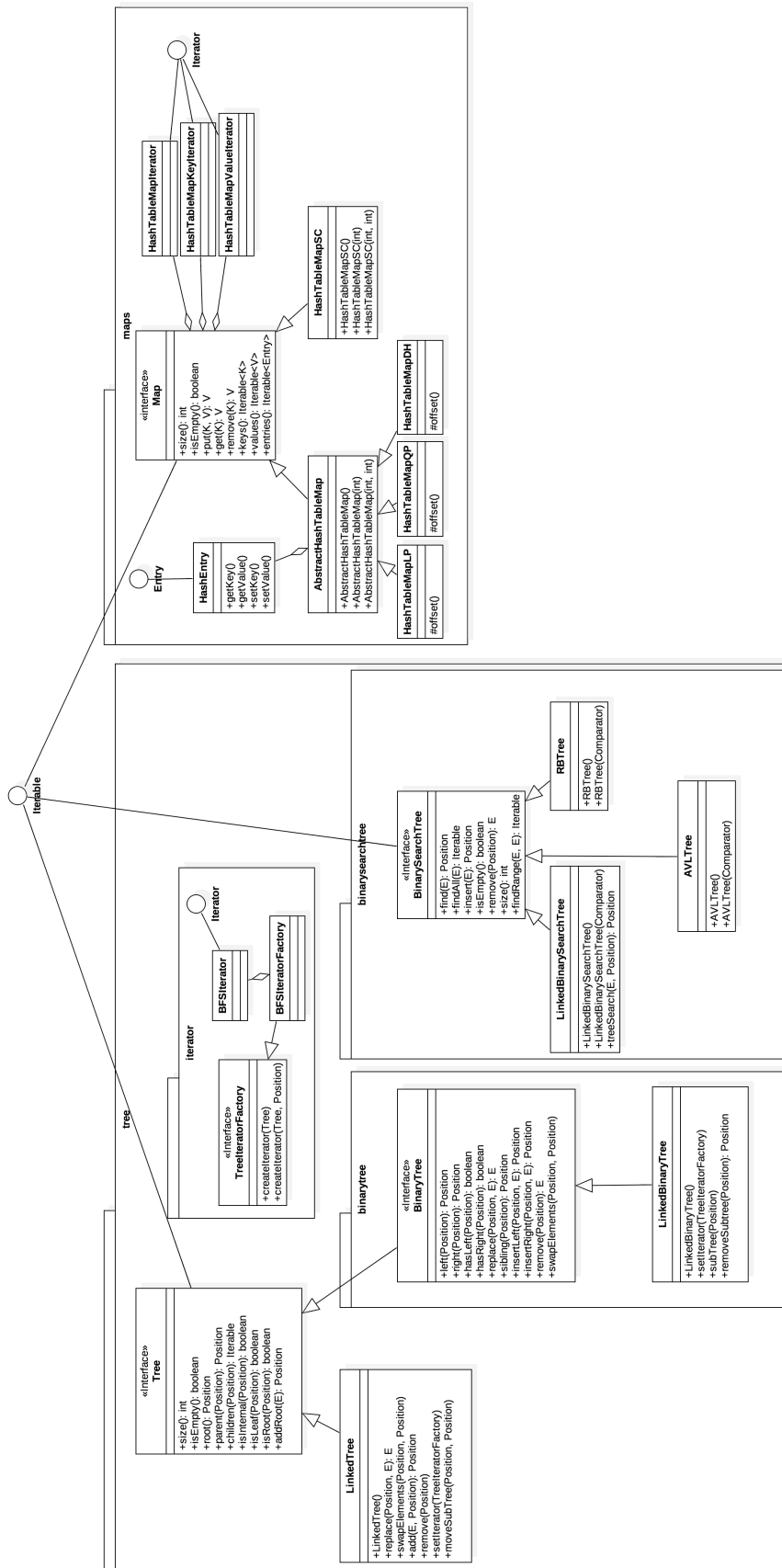
A continuación se proporciona un fragmento de código con el que ilustra el uso de esta estructura de datos.

```
public void test ElectoralCollege () {
    ElectoralCollege instance = new ElectoralCollege();
    instance.addVoter("111111111A", "c/ Clavel", "Juan Perez Perez");
    instance.addVoter("222222222A", "c/ Petunia", "Andres Martin Palomo");
    instance.addVoter("333333333B", "c/ Lila", "Pedro Serrano Vega");
    instance.addVoter("444444444C", "c/ Leonidas", "Daniel Vela Amor");

    instance.makeStationDistribution();

    ..... resultA = instance.getAllVoters();
    ..... resultB = instance.getStationVoters('A');
    ....
}
```

NOTA: En la clases suministradas se podrán agregar los métodos que se consideren oportunos y cualquiera de las estructuras de datos estudiadas durante el curso, valorándose especialmente lo adecuado de la elección que se realice.



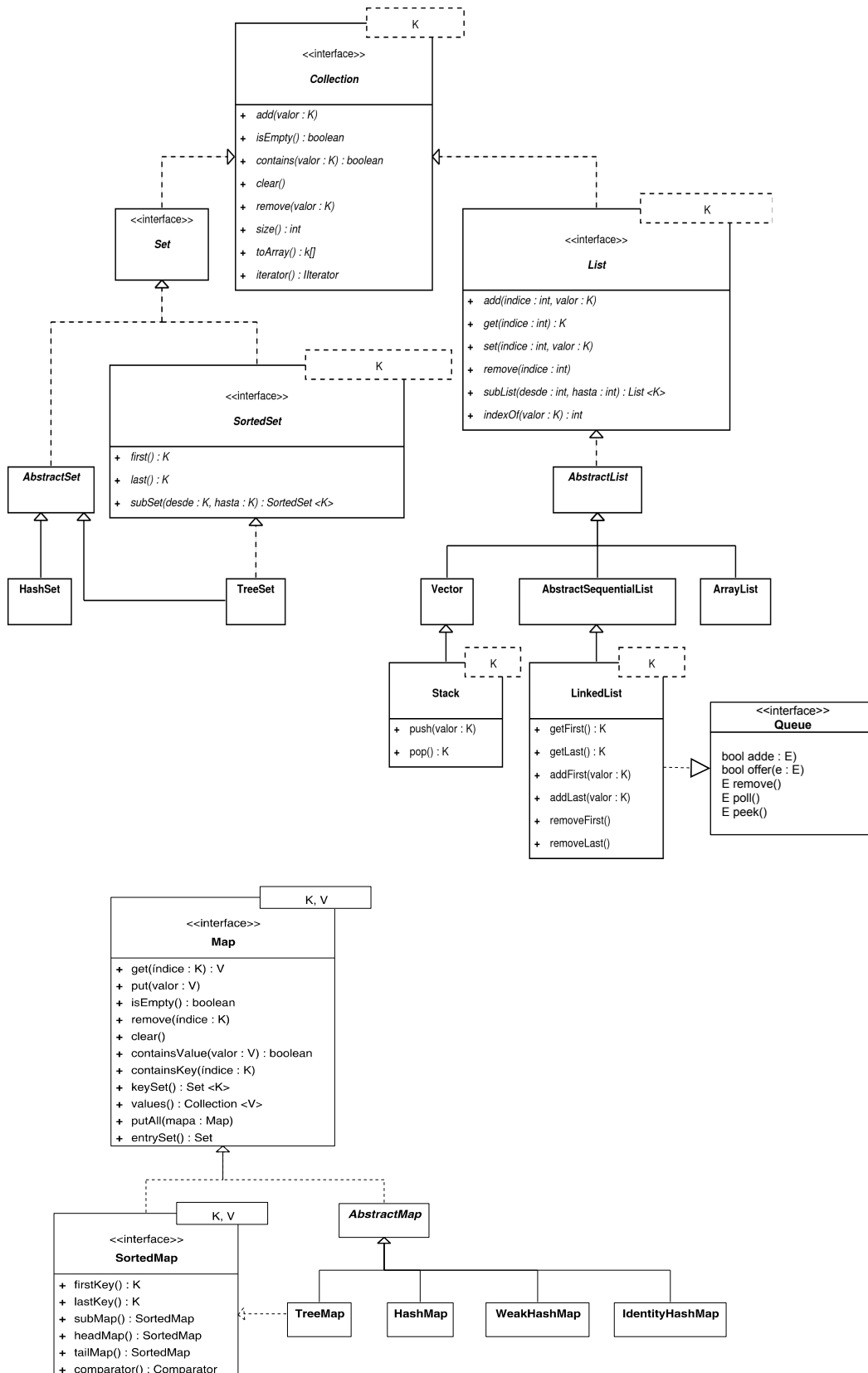


Diagrama de clases correspondiente a las estructuras propias de Java



Universidad
Rey Juan Carlos

CENTRO:

TITULACION:

ASIGNATURA:

D.N.I.:

APELLIDOS:

NOMBRE:

FECHA:

CURSO:

GRUPO:

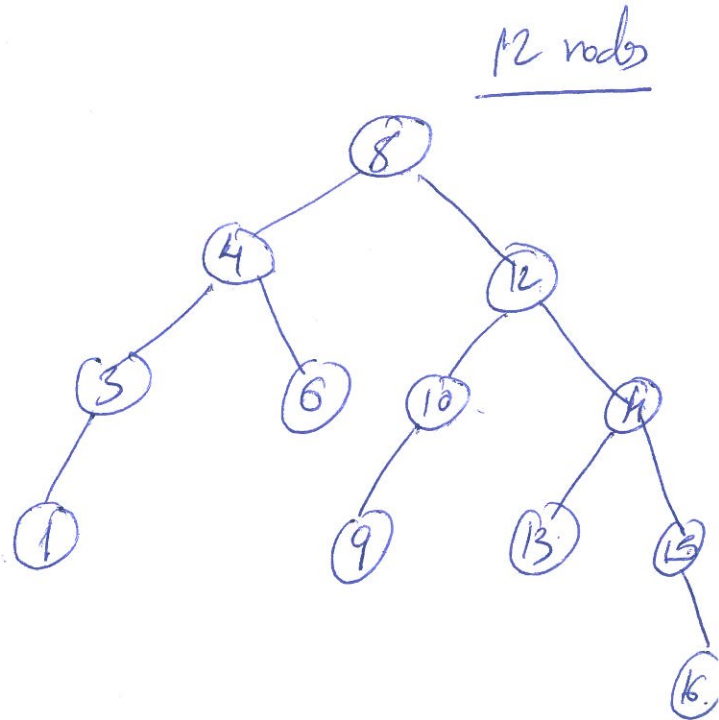
Ejercicio 1.a

AVL

Paso 1: Rame con prof 5

Paso 2: Añadir nodes hasta
que esté equilibrado

Paso 3: Asignar valores.



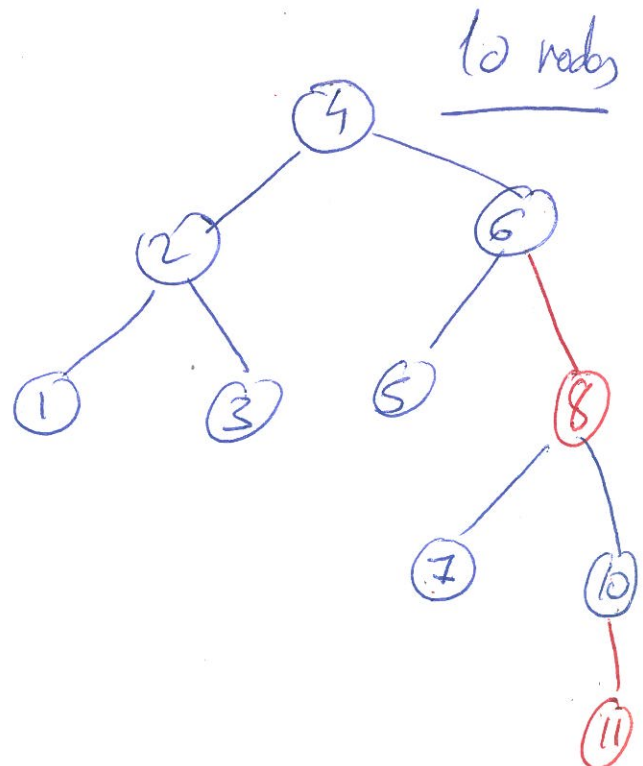
Kojo - Negro

Paso 1: $h=5 \Rightarrow hb=3(2r+3n)$

Paso 2: Dibujar la rama más
larga (n-r-n-r-...)

Paso 3: Dibujar la rama más
corta (n-n-n-...)

Paso 4: Equilibrar.



Ejercicio 1b

El factor de carga en mapas define ~~la~~ eficacia α
se define como

$$\alpha = \frac{\text{num. entres}}{\text{tam. tabl}}$$

En el caso de encadenamiento separado el valor pred
valor como máximo 0.75 por lo tanto, si queremos meter
50 entres

$$0.75 = \frac{50}{\text{tam. tabl}} \Rightarrow \text{tam. tabl} = 66.67 \Rightarrow \underline{\underline{67}}$$

En direccionamiento abierto el máximo es $\alpha = 0.5$
lo que implica que el tamaño mínimo para 50
entres $\Rightarrow$

$$0.5 = \frac{50}{\text{tam. tabl}} \Rightarrow \text{tam. tabl} = \underline{\underline{100}}$$

Ejercicio 1c

Orden 7 $\Rightarrow$ 6 claves por nodo

Nivel 1: 1 nodo $\Rightarrow$ 6

Nivel 2: 7 nodos $\times$ 6 claves $\Rightarrow$ 42

Nivel 3: $7 \times 7 \times 6 = 294$

```

package examenenes.junio2016;
import java.util.Comparator;
import material.tree.LinkedTree;
import material.tree.Position;
import material.tree.binarysearchtree.LinkedBinarySearchTree;

public class Ejercicio2<E> extends LinkedBinarySearchTree<E> {
    // Ejercicio 2.1
    private void buildTree(Position<E> binTreeNode, LinkedTree<E> linkedTree, Position<E> linkedTreeNode) {
        for (Position<E> child : binTreeNode.children(binTreeNode)) {
            if (child.getElement() != null) {
                Position<E> newNode = linkedTree.add(child.getElement(), linkedTreeNode);
                buildTree(child, linkedTree, newNode);
            }
        }
    }

    public LinkedTree toLinkedTree() {
        LinkedTree<E> linkedTree = new LinkedTree<>();
        if (!binTree.isEmpty()) {
            linkedTree.addRoot(binTree.root().getElement());
            buildTree(binTree.root(), linkedTree, linkedTree.root());
        }
        return linkedTree;
    }

    //Ejercicio 2.2.
    public Position<E> findLargerOrEqual(E min) {
        Position<E> p = treeSearch(min, binTree.root());

        while ((p.getElement() == null) || (comparator.compare(p.getElement(), min) < 0)) {
            p = binTree.parent(p);
        }
        return p;
    }

    public void removeRange(E min, E max) {
        Position<E> p = findLargerOrEqual(min);

        boolean inRange = ((comparator.compare(p.getElement(), min) >= 0) && (comparator.compare(p.getElement(), max) <= 0));
        while ((p.getElement() != null) && inRange) {
            remove(p);
            p = findLargerOrEqual(min);
            inRange = ((comparator.compare(p.getElement(), min) >= 0) && (comparator.compare(p.getElement(), max) <= 0));
        }
    }

    // Ejercicio 2.c. Opción 1
    public class Student implements Comparable<Student> {
        private String name;
        private String surname;
        private int exp;

        public Student(String n, String s, int e) {
            this.name = n;
            this.surname = s;
            this.exp = e;
        }

        @Override
        public int compareTo(Student o) {
            int res = this.name.compareTo(o.name);
            if(res == 0){
                res = this.surname.compareTo(o.surname);
            }
            return res;
        }
    }

    // Ejercicio 2.c. Opción 2
    public class StudentComparator implements Comparator<Student>{
        @Override
        public int compare(Student o1, Student o2) {
            int res = o1.name.compareTo(o2.name);
            if(res == 0){
                res = o1.surname.compareTo(o2.surname);
            }
            return res;
        }
    }
}

```

```

package examenes.junio2016;

import java.util.ArrayList;
import java.util.List;
import material.maps.HashMapLP;
import material.maps.Map;
import material.orderddictionary.AVLOrderedDict;
import material.orderddictionary.OrderedDictionary;

public class ElectoralCollege {

    final int maxPerStation = 800;

    OrderedDictionary <String,String> votersInCollege;
    Map <Character,List <String>> votersInStation;
    List <VoterInStation> fullList;

    public ElectoralCollege() {
        votersInCollege = new AVLOrderedDict<>();
        votersInStation = new HashMapLP<>();
        fullList = new ArrayList<>();
    }

    /**
     * Método que permite añadir los datos de un votante a las estructuras de datos consideradas.
     * @param DNI Cadena de 9 caracteres con el DNI de un votante.
     * @param street Cadena de longitud indeterminada con la calle del votante.
     * @param fullName Cadena de longitud indeterminada con el nombre completo del votante.
     */
    public void addVoter(final String DNI, final String street, final String fullName) {
        votersInCollege.insert(DNI, fullName + ", " + street);
    }

    /**
     * @return Devuelve una lista de votantes y la mesa a la que pertenecen ordenadas por DNI.
     */
    public Iterable<VoterInStation> getAllVoters() {
        return fullList;
    }

    /**
     * Método que realiza la distribución en mesas electorales de los votantes introducidos.
     * Los votantes se reparten entre las mesas de manera ordenada. La mesa A tiene
     * los votantes cuyo número es más bajo, la mesa B tiene los siguientes en numeración, y así
     * sucesivamente.
     */
    public void makeStationDistribution() {
        final int voterPerStation = votersInCollege.size()/maxPerStation;

        char firstLetter = 'A';
        int cont = 0;
        for (String DNI : votersInCollege.keys()) {
            if (cont > voterPerStation) {
                cont = 0;
                firstLetter++;
            }

            if (votersInStation.get(firstLetter) == null) {
                votersInStation.put(firstLetter, new ArrayList<>());
            }
            votersInStation.get(firstLetter).add(DNI + ", " + votersInCollege.find(DNI));
            fullList.add(new VoterInStation(DNI, firstLetter));
        }
    }

    /**
     * @param station Letra mayúsculas de la mesa electoral.
     * @return Devuelve una lista con el DNI, nombre completo y calle de los votantes en una mesa determinada.
     * Devuelve null si no hay esa mesa.
     */
    public Iterable<String> getStationVoters(char station) {
        return votersInStation.get(station);
    }
}

```

```

package examenes.junio2016;

import java.util.Objects;

/**
 * Clase que representa la asignación de un votante y una mesa
 */

public class VoterInStation {

    private final String DNI;
    private final char station;

    public VoterInStation(final String DNI, final char station) {
        this.DNI = DNI;
        this.station = station;
    }

    public String getDNI() {
        return DNI;
    }

    public char getStation() {
        return station;
    }

    @Override
    public int hashCode() {
        int hash = 7;
        hash = 23 * hash + Objects.hashCode(this.DNI);
        hash = 23 * hash + this.station;
        return hash;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null) {
            return false;
        }
        if (getClass() != obj.getClass()) {
            return false;
        }
        final VoterInStation other = (VoterInStation) obj;
        if (this.station != other.station) {
            return false;
        }
        return Objects.equals(this.DNI, other.DNI);
    }
}

```